



MIAGE- L3

Systèmes Informatiques

Système d'exploitation

Entrées/Sorties

BAKAYOKO Ibrahima

Enseignant-chercheur en informatique

UFR Mathématiques & Informatique

Bakayoko.ibrahima1@ufhb.edu.ci

Open Our Mind for a free world

Imaginez un ordinateur comme une ville animée, avec ses habitants, ses rues et ses bâtiments. Le système d'exploitation est le maire de cette ville, responsable de maintenir l'ordre, de gérer les ressources, et de s'assurer que tout fonctionne harmonieusement. Pour ce faire, le maire doit constamment gérer les flux de personnes et de marchandises entrant et sortant de la ville.

Les entrées-sorties (E/S) représentent les interactions entre la ville (ordinateur) et le monde extérieur. Ces interactions peuvent prendre de nombreuses formes : des données qui entrent depuis un clavier ou une souris, des données qui sortent vers un écran ou une imprimante, ou même des données qui voyagent via un réseau vers d'autres ordinateurs.

Objectif général

- Comprendre en profondeur le rôle des périphériques **d'entrées/sorties**, le processus **d'ordonnancement** de ces opérations et les problématiques liées aux "**deadlocks**" dans les systèmes d'exploitation.

Objectifs spécifiques 1/3

- Comprendre le rôle des périphériques d'entrées/sorties dans les systèmes informatiques.
- Identifier les types courants de périphériques d'entrées/sorties.
- Expliquer les mécanismes de communication entre le système d'exploitation et les périphériques d'entrées/sorties.

Objectifs spécifiques 2/3

- Comprendre l'importance de l'ordonnancement des entrées/sorties pour les performances du système.
- Étudier quelques algorithmes d'ordonnancement des entrées/sorties.
- Expliquer comment l'ordonnancement des entrées/sorties améliore l'efficacité et l'équité d'accès aux ressources.

Objectifs spécifiques 3/3

- Comprendre ce qu'est un "deadlock" et ses conséquences dans un système informatique.
- Identifier les conditions nécessaires à la formation d'un deadlock.
- Expliquer les différentes stratégies de gestion des deadlocks.

PLAN

INTRODUCTION

I. LES PÉRIPHÉRIQUES D'ENTRÉES/SORTIES

II. ORDONNANCEMENT DES ENTRÉES/SORTIES

III. LES DEADLOCKS

CONCLUSION

LES PÉRIPHÉRIQUES D'ENTRÉES/SORTIES

LES PÉRIPHÉRIQUES D'ENTRÉES/SORTIES 1/2

Périphériques	Débits
Clavier	10 bits/s
Souris	100 bits/s
Modem 56K	7 kb/s
Scanneur	400 kbits/s
caméscope numérique	4 MB/s
CD-ROM 52x	8 MB/s
Firewire (IEEE1394)	50 MB/s

Périphériques	Débits
USB 2.0	60 bits/s
Moniteur XGA(Extended Graphics Array)	60 bits/s
SONET oc-12 network (Synchronous Optical Network)	78 MB/s
Gigabit ethernet	125 MB/s
Serial ATA DISK (Disque Serial ATA ou SATA)	200 MB/s
Disque SCSI Ultrawide 4 (Small Computer System Interface)	320 MB/s
Bus PCI	528 MB/s

DEUX SORTES DE PÉRIPHÉRIQUE

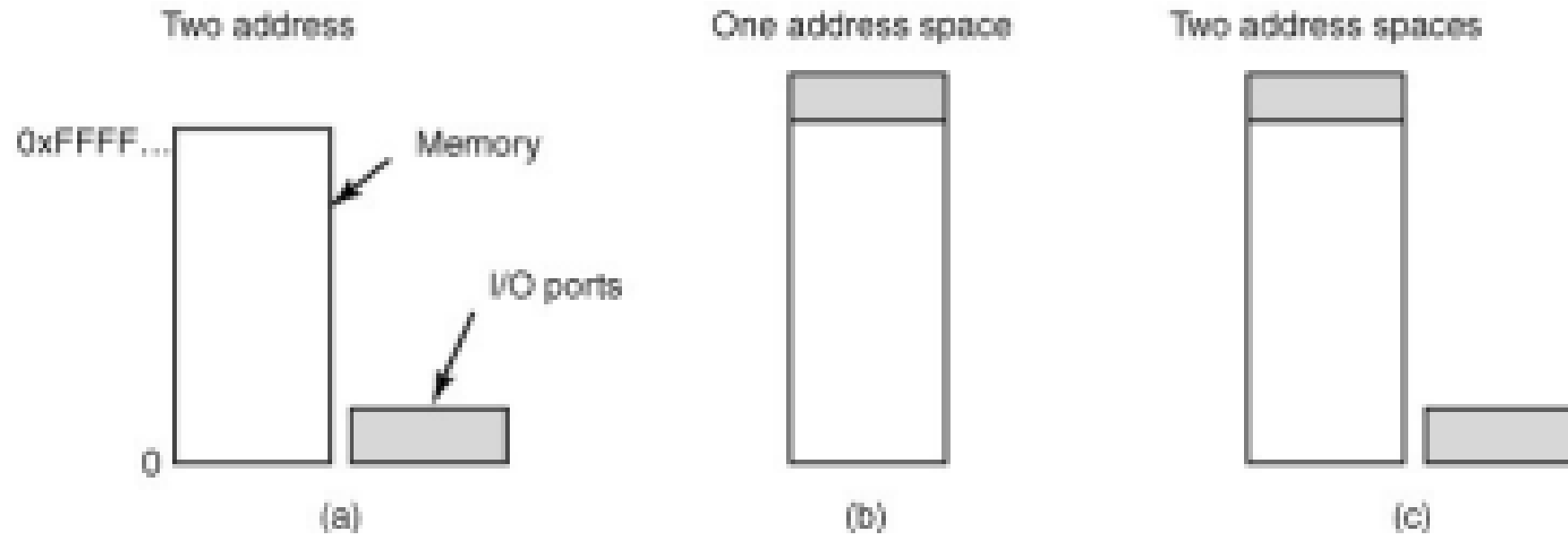
- ❑ Les **block devices** manipulent les blocs de taille fixe
 - ❑ Exemple : disques (1 secteur = 512 octets)
- ❑ Les **character devices** manipulent des données de n'importe quelle taille
 - ❑ *Claviers*
 - ❑ *souris*
 - ❑ *Imprimantes*
 - ❑ *Cartes réseau*
 - ❑ *Etc.*

- ❑ Tous les périphériques sont différents les uns des autres
 - ❑ Partie mécaniques etc.
- ❑ Chaque périphérique contient un **contrôleur** en charge de communiquer avec le reste de la machine
 - ❑ *Envoie de commandes au bras du disque*
 - ❑ *Convertit les bits en blocs*
 - ❑ *Etc.*

- ❑ Les contrôleurs peuvent être très sophistiqués
 - ❑ *Buffering*
 - ❑ *Exécution spéculative*
 - ❑ *Etc.*

La mémoire mappée, également connue sous le nom de mémoire mappée dans un fichier, est une technique informatique qui permet d'accéder aux fichiers sur un support de stockage (généralement un disque dur) en les associant directement à l'espace d'adressage mémoire d'un programme informatique.

- ❑ Chaque contrôleur a des registres mémoire internes
 - ❑ Le CPU lit/écrit dans ces registres (grâce au bus) pour communiquer avec le périphérique
- ❑ Une abstraction utile : prétendons que **ces registres font partie de l'espace d'adressage normal de la machine**
 - ❑ Il faut réserver un intervalle d'adresses mémoire (hors de la mémoire physique de la machine)
 - ❑ Pour communiquer avec le périphérique, le CPU doit simplement lire/écrire dans ces adresses mémoire
 - ❑ Le bus s'occupe du reste



Fonctionnement de la mémoire mappée :

Association avec la mémoire : Lorsqu'un fichier est ouvert en mode mémoire mappée, une partie ou la totalité du fichier est associée à l'espace d'adressage mémoire du processus qui l'a ouvert. Cela signifie que le contenu du fichier est directement accessible en mémoire.

Accès direct : Une fois que le fichier est mappé en mémoire, il peut être accédé directement comme s'il s'agissait d'une partie de la mémoire vive. Cela signifie que vous pouvez lire et écrire dans le fichier en utilisant des pointeurs mémoire et des opérations de lecture/écriture standard plutôt que des appels de lecture/écriture de fichiers distincts.

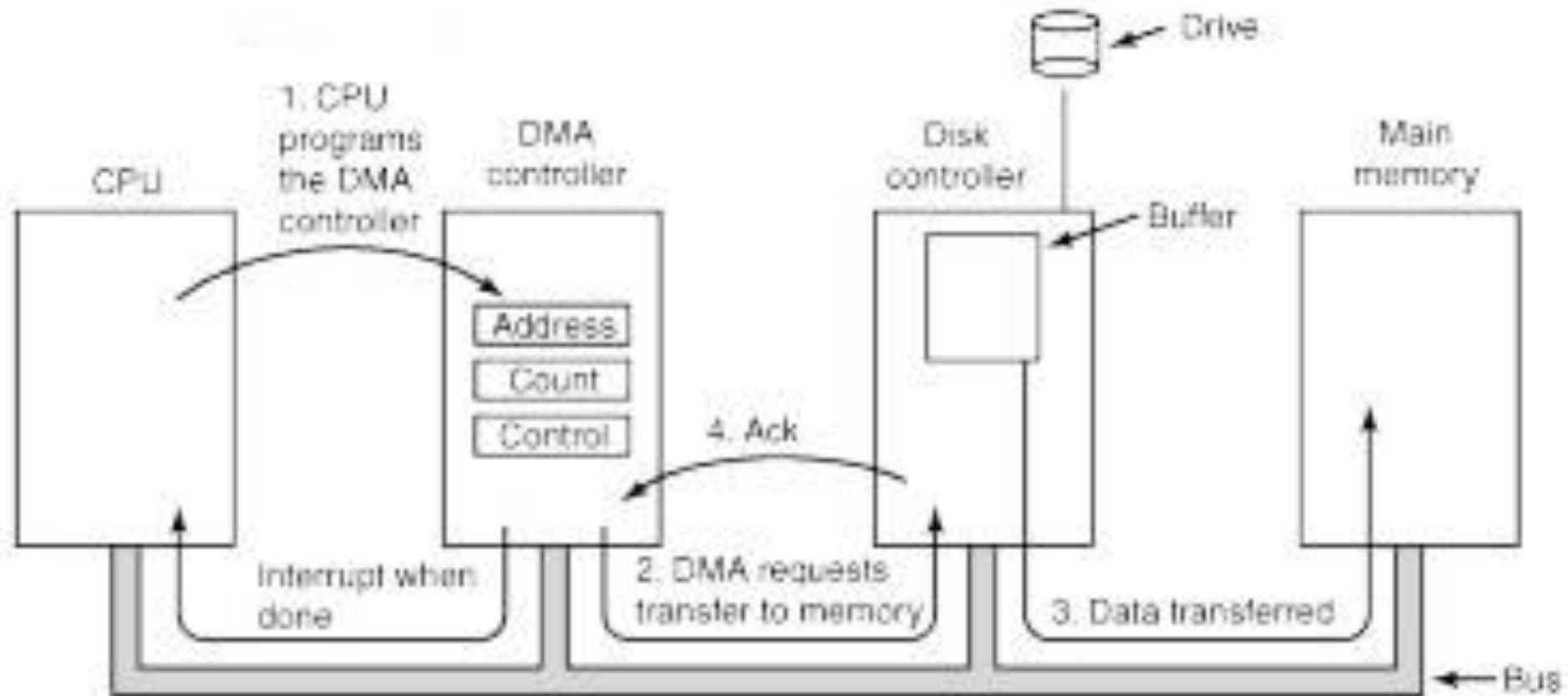
Optimisation des E/S (Entrée/Sortie) : La mémoire mappée est souvent utilisée pour optimiser les opérations d'E/S, car elle peut réduire les opérations coûteuses de copie de données entre la mémoire et le stockage. Lorsque vous lisez ou écrivez dans une zone de mémoire mappée, le système d'exploitation s'occupe de transférer les données entre la mémoire et le disque de manière transparente.

Partage de données : La mémoire mappée permet également de partager des données entre plusieurs processus. Si plusieurs processus ouvrent le même fichier en mode mémoire mappée, ils peuvent partager les mêmes données en mémoire, ce qui peut être utile dans certaines situations de programmation.

C'est une technique utilisée dans les SI pour permettre à certains périphériques matériels, tels que des contrôleurs de disque dur, des cartes graphiques ou des cartes réseau, d'accéder directement à la mémoire principale (RAM) d'un ordinateur sans passer par le CPU. Cela permet d'améliorer les performances du système en réduisant la charge du CPU lors du transfert de données entre les périphériques et la mémoire.

C'est une technique utilisée dans les SI pour permettre à certains périphériques matériels, tels que des contrôleurs de disque dur, des cartes graphiques ou des cartes réseau, d'accéder directement à la mémoire principale (RAM) d'un ordinateur sans passer par le CPU. Cela permet d'améliorer les performances du système en réduisant la charge du CPU lors du transfert de données entre les périphériques et la mémoire.

- ❑ Les registres du contrôleur font partie de l'espace d'adressage uniforme de la machine
 - ❑ Mais ce n'est qu'une abstraction. Il faut quand même transférer les données à travers le bus
- ❑ DMA = permettre aux périphériques de parler directement avec la mémoire
 - ❑ Sans devoir transiter par le CPU
 - ❑ Quand le transfert est terminé : le contrôleur DMA envoie une interruption au CPU



Fonctionnement DMA :

Initialisation du DMA : Le système d'exploitation ou le matériel configure initialement le contrôleur DMA en indiquant quelles adresses mémoire il peut accéder et quelles opérations il est autorisé à effectuer. Ces paramètres sont configurés pour éviter que le DMA n'écrase accidentellement des données importantes en mémoire.

Demande de transfert DMA : Lorsqu'un périphérique a besoin de transférer des données vers ou depuis la mémoire, il envoie une demande au contrôleur DMA. Cette demande peut prendre la forme d'une interruption matérielle ou d'un signal spécial.

Validation par le CPU : Le CPU doit généralement autoriser le transfert DMA en vérifiant que la demande provient d'un périphérique légitime et qu'elle n'est pas malveillante. Une fois que le CPU a validé la demande, il peut lancer le transfert DMA.

Transfert de données : Le contrôleur DMA, une fois autorisé, accède directement à la mémoire principale et transfère les données entre le périphérique et la mémoire sans impliquer le CPU. Cela permet au CPU de continuer à exécuter d'autres tâches pendant le transfert, améliorant ainsi l'efficacité globale du système.

Interruption et notification : Une fois le transfert terminé, le contrôleur DMA peut générer une interruption pour notifier le CPU que l'opération est terminée. Le CPU peut alors prendre les mesures appropriées, telles que la gestion des données transférées.

1. Indépendance du matériel

- Lire des données d'un CD-ROM, d'un disque ou d'une clé USB doit fonctionner de la même façon

2. Nommage uniforme

- Tous les périphériques doivent avoir des noms uniformes
- Par exemple : /media/cdrom/repertoire/fichier.pdf

3. Gestion d'erreurs

- Gérer les erreurs aussi près du hardware que possible
- Ne pas déranger les couches hautes sauf si c'est indispensable

4. Entrées/sorties synchrones ou asynchrones

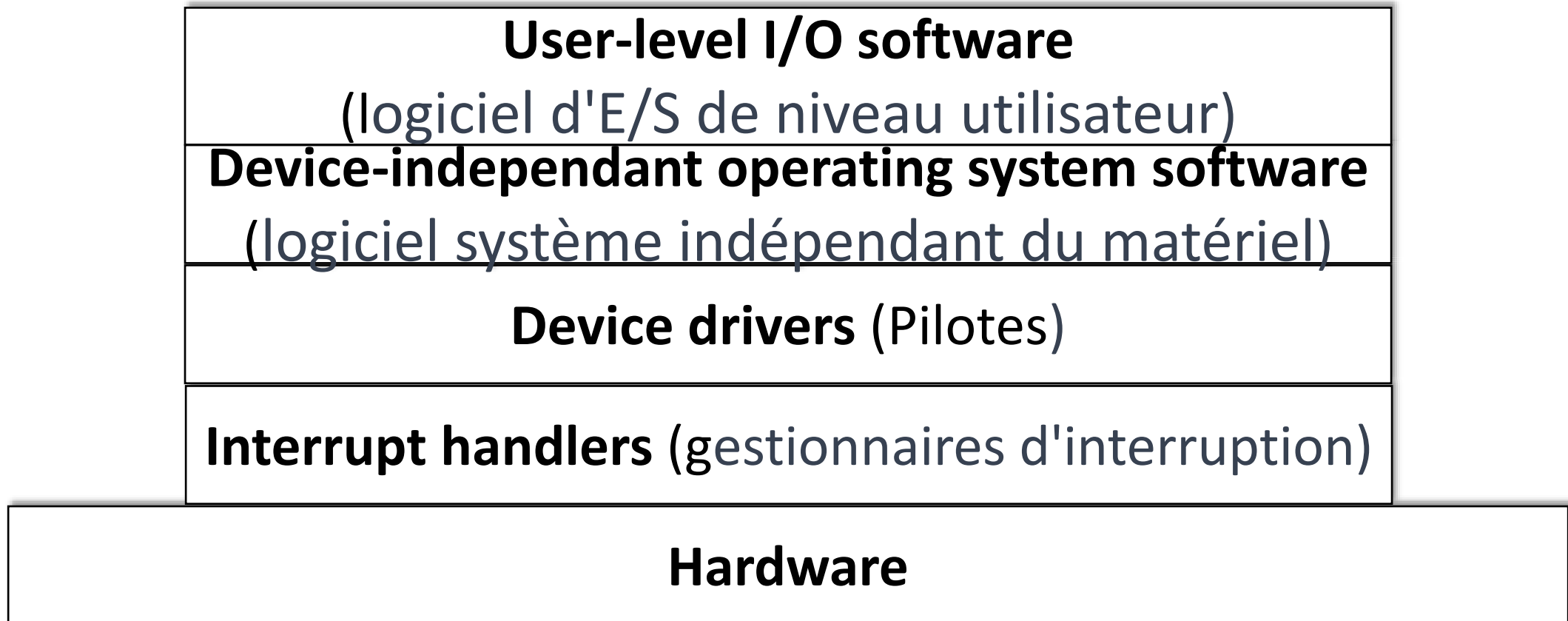
- Toutes les E/S sont asynchrones : le CPU peut faire autre chose en attendant
- Mais les programmes utilisateurs préfèrent souvent l'**illusion** que les E/S sont synchrones

5. Buffering

- Souvent on ne peut pas choisir le **moment** où les données arrivent d'un périphérique
- Exemple : carte réseau
- Il faut stocker les données en attendant qu'elles soient consommées

Organisation du système d'entrées/sorties

- Le système d'entrées/sorties est généralement organisé en **quatre couches**



Le fonctionnement du système d'entrées/sorties (E/S), également appelé système d'E/S, concerne la manière dont un ordinateur interagit avec les périphériques et gère les opérations d'entrée (comme la réception de données depuis des dispositifs externes) et de sortie (comme l'envoi de données vers des dispositifs externes).

Voici un aperçu simplifié de son fonctionnement :

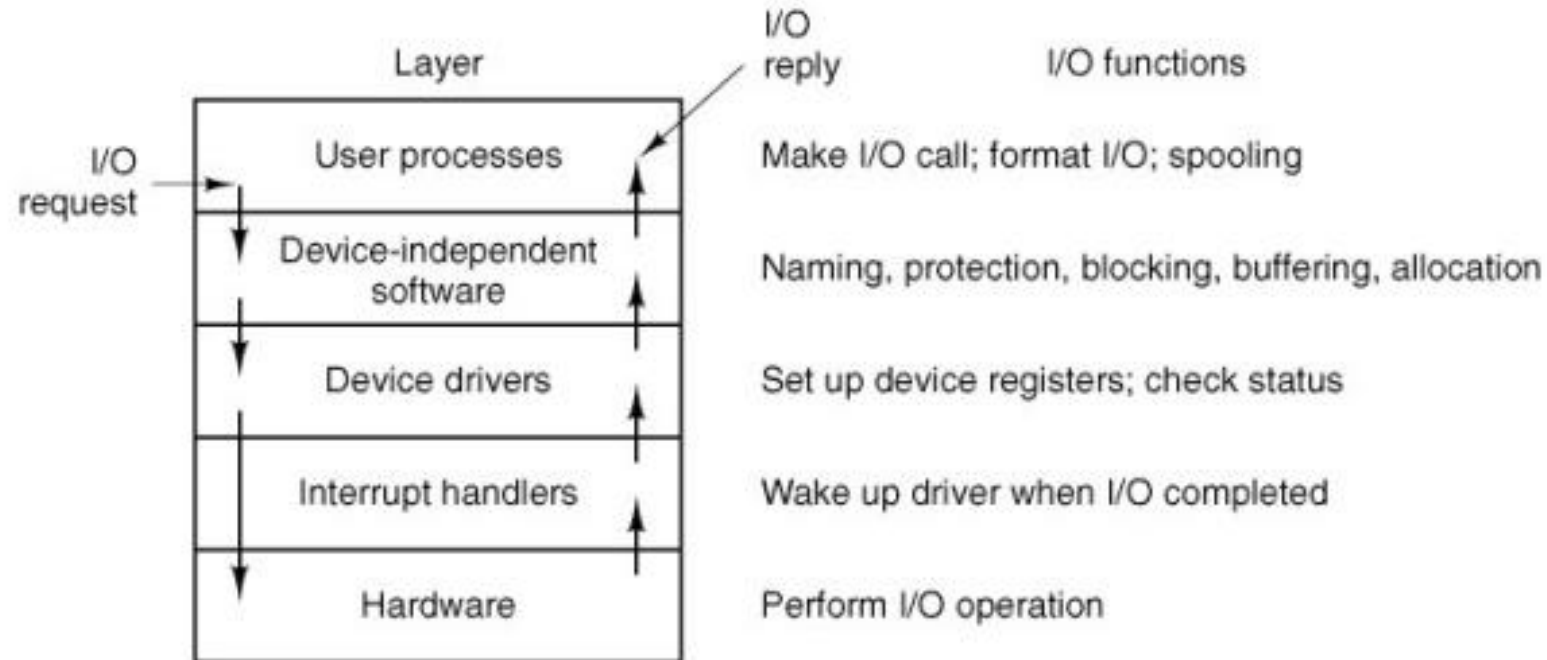
1. Requêtes d'E/S : Lorsqu'un programme souhaite effectuer une opération d'entrée/sortie, il fait une demande au système d'exploitation en utilisant des appels système ou des bibliothèques spécifiques. Par exemple, il peut demander de lire un fichier ou d'envoyer des données à une imprimante.

- 2. Gestion des requêtes :** Le système d'exploitation reçoit ces requêtes et les organise dans une file d'attente (file d'attente d'E/S) en fonction de leur priorité et de l'ordre d'arrivée.
- 3. Allocation des ressources :** Le système d'exploitation alloue les ressources nécessaires à chaque opération d'E/S. Cela peut impliquer la gestion des accès concurrents aux périphériques, la réservation de mémoire tampon, etc.

4. Exécution des opérations : Le système d'exploitation contrôle l'accès aux périphériques physiques et s'assure que les données sont transférées correctement entre la mémoire et les périphériques. Cela peut nécessiter la programmation des contrôleurs matériels spécifiques.

5. Gestion des interruptions : Lorsque l'opération d'E/S est terminée ou nécessite une action particulière, le périphérique envoie une interruption au processeur. Le système d'exploitation doit gérer ces interruptions et réagir en conséquence, en passant souvent le contrôle à un gestionnaire d'interruption (interrupt handler) approprié.

- 6. Notification au programme utilisateur :** Une fois que l'opération d'E/S est terminée, le système d'exploitation informe le programme utilisateur que l'opération a été effectuée avec succès ou qu'une erreur s'est produite.
- 7. Libération des ressources :** Une fois que l'opération d'E/S est terminée et que les données ont été correctement transférées, le système d'exploitation libère les ressources associées à cette opération.

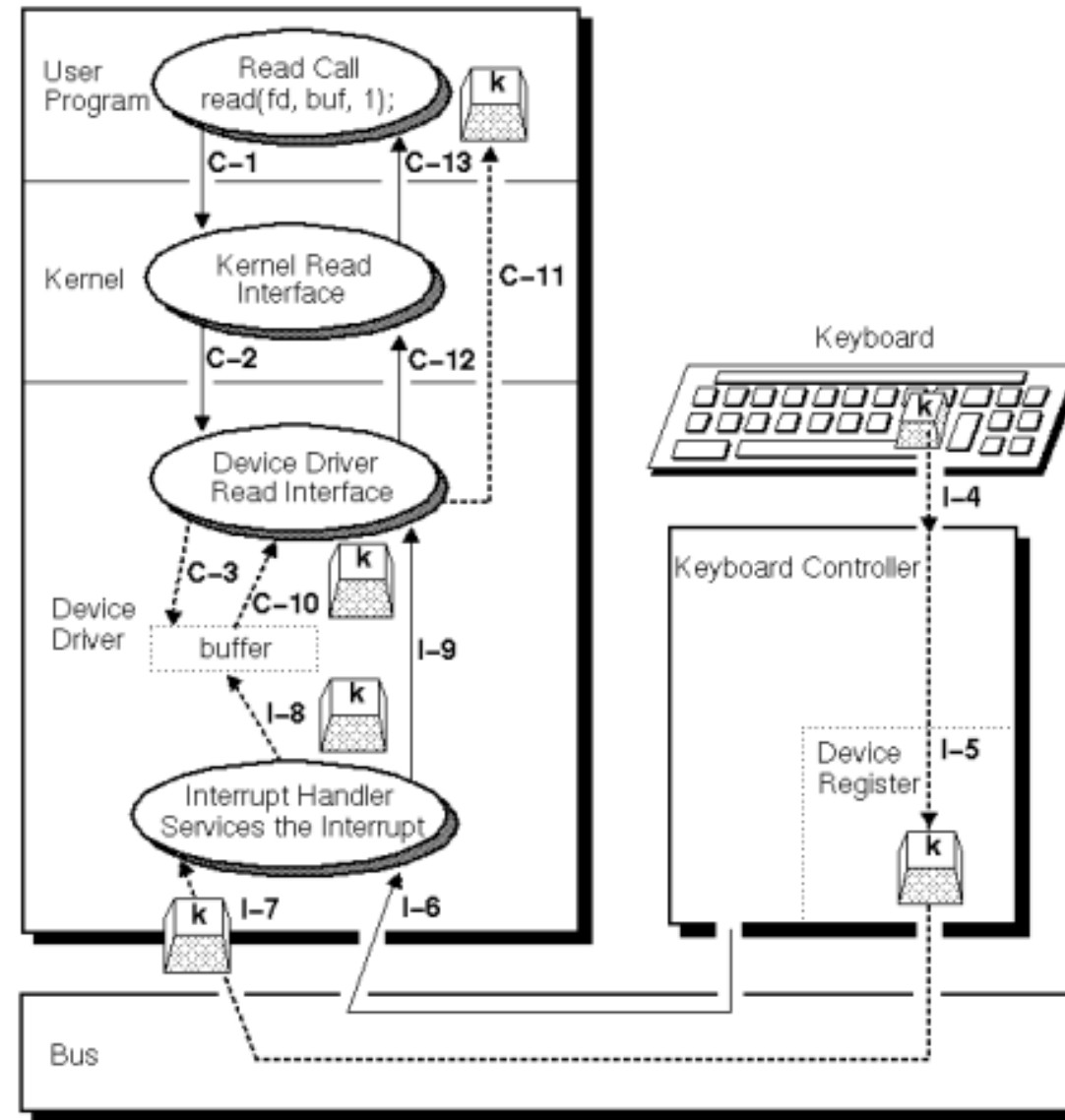


- Programmer à base d'interruptions est **difficile**
 - Votre programme peut être interrompu à n'importe quel moment
 - Il faut maintenir le contexte de chaque opération en cours dans le système

- Plus facile pour les utilisateurs : rendre les entrées-sorties synchrones (artificiellement)
 - Quand un programme demande une entrée/sortie : bloquer le driver avec un sémaphore
 - Quand on reçoit l'interruption : débloquent le driver en débloquent le sémaphore C'est plus facile à implémenter si chaque driver de périphérique est implémenté sous forme d'un processus séparé

- Les périphériques sont **très différents les uns des autres**
 - Un driver de souris reçoit des informations de mouvement, de distance, de direction, de clics etc.
 - Un driver de disque manipule des secteurs, des pistes, des cylindres, etc. cylinders, heads, arm motion, motor drives, etc.
 - Même deux périphériques similaires (p.ex. deux cartes réseaux) peuvent fonctionner très différemment et doivent recevoir des commandes différentes

- Comment le système sait-il comment utiliser chaque périphérique ?
 - Chaque fabricant écrit un **device driver** qui sait comment utiliser le périphérique
 - Chaque driver implémente l'API standard le mieux possible
Grâce à la connaissance du fonctionnement interne du périphérique...



Description simplifiée :

1. **Attente d'une Entrée Utilisateur** : Le programme attend que l'utilisateur entre un caractère via le clavier.
2. **Saisie d'un Caractère** : L'utilisateur appuie sur une touche du clavier, ce qui génère un signal électrique.
3. **Signal Électrique** : Le signal électrique est transmis à la carte mère de l'ordinateur.

4. **Conversion en Code** : La carte mère convertit le signal en un code numérique correspondant au caractère saisi.
5. **Buffer d'Entrée** : Le système d'exploitation a un buffer d'entrée (une zone de mémoire) pour stocker ce code.
6. **Lecture du Buffer** : Le programme peut lire ce code depuis le buffer d'entrée pour déterminer quel caractère a été saisi.

7. **Traitement du Caractère** : Le programme peut effectuer des actions en fonction du caractère saisi. Par exemple, l'afficher à l'écran ou l'utiliser pour une opération particulière.
8. **Répétition** : Ce processus se répète continuellement, ce qui permet au programme de répondre en temps réel aux saisies de l'utilisateur.

- Il faut un **nom** à chaque périphérique
- Dans Unix : chaque périphérique est représenté par un **fichier spécial** :
 - /dev/hda, /dev/hdb représentent des disques IDE/SATA
 - /dev/sda, /dev/sdb représentent des disques SCSCI/USB(ex. clé USB)
 - Etc.

- On peut créer d'autres **fichiers spéciaux**
 - /dev/null/, /dev/random, etc.
- Chaque périphérique est décrit par deux nombres :
 - **Major device number** : définit le type de périphérique(cela permet de choisir le bon clavier)
 - **Minor device number** : indique un périphérique particulier
- Donner des noms de fichiers aux fichiers aux périphériques permet également de leur donner des **droits d'accès**

PLAN

INTRODUCTION

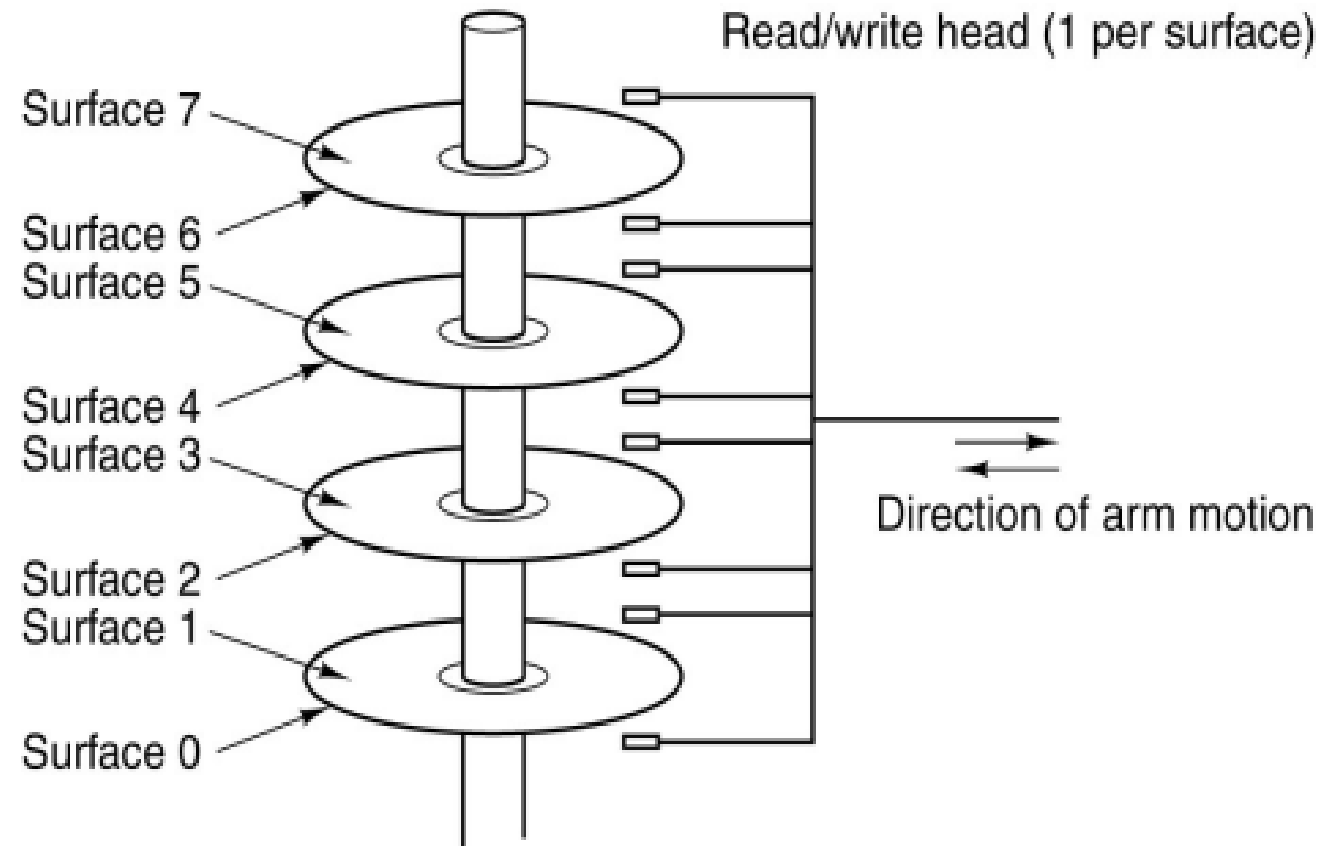
I. LES PÉRIPHÉRIQUES D'ENTRÉES/SORTIES

II. ORDONNANCEMENT DES ENTRÉES/SORTIES

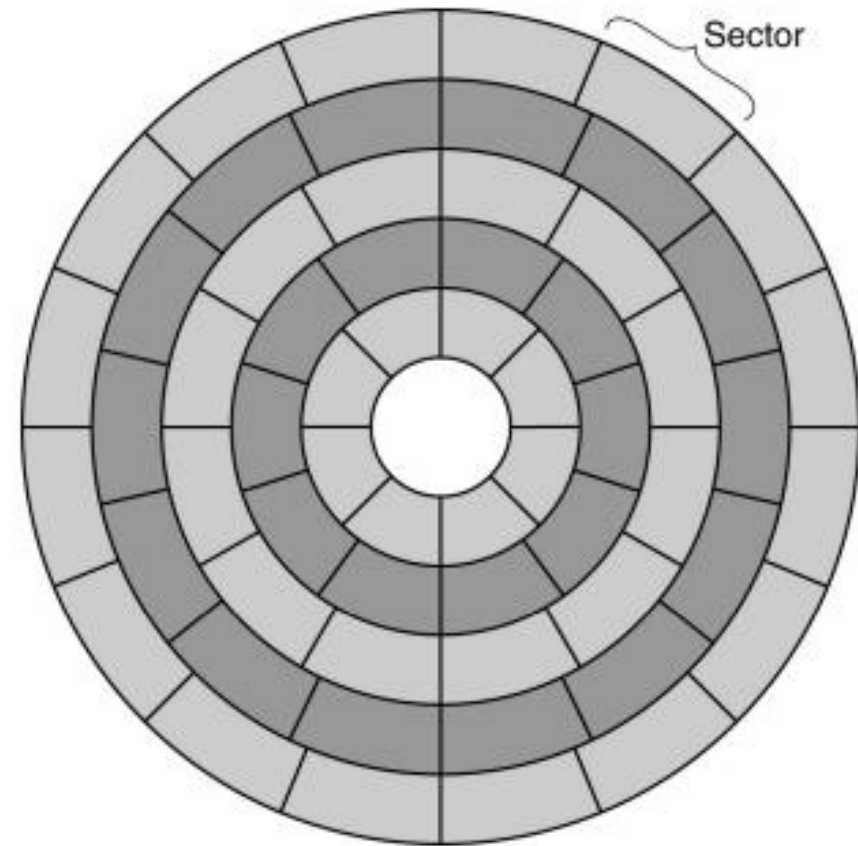
III. LES DEADLOCKS

CONCLUSION

- Un disque dur est composé de :
 - 6-12 plateaux (qui tournent à 5400-10000 rotations/min)
 - 1 bras et une tête pour lire/écrire les données

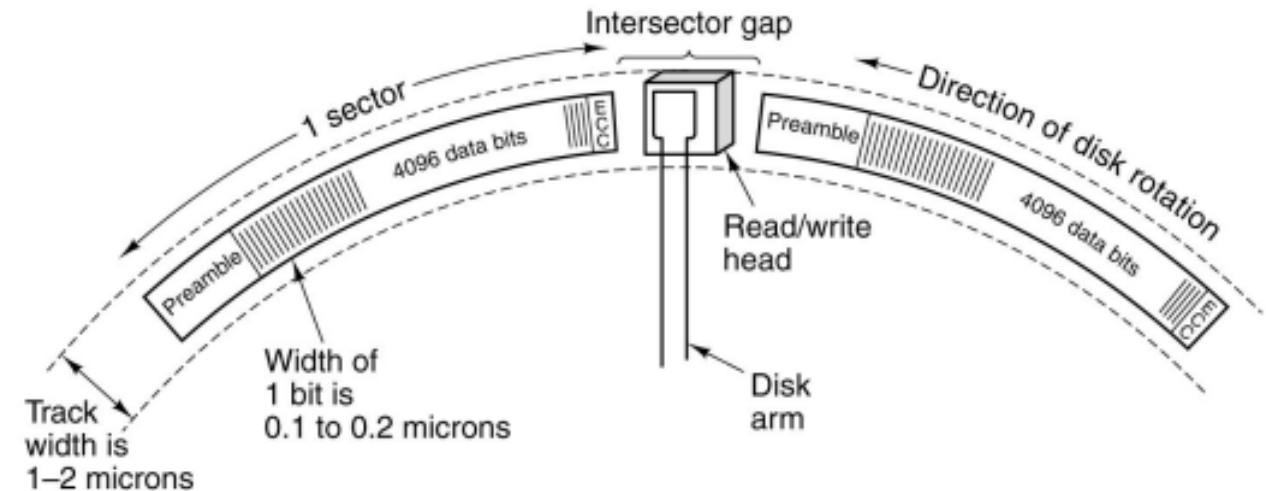


- Chaque plateau contient **10-30 zones**
- Chaque zone contient **plusieurs secteurs** (les zones extérieures ont plus de secteurs que les zones intérieures)



- Chaque plateau contient :

- Un préambule (pour permettre à la tête de reconnaître le début du secteur)
- 512 octets de données
- Un code de correction d'erreur(ECC)

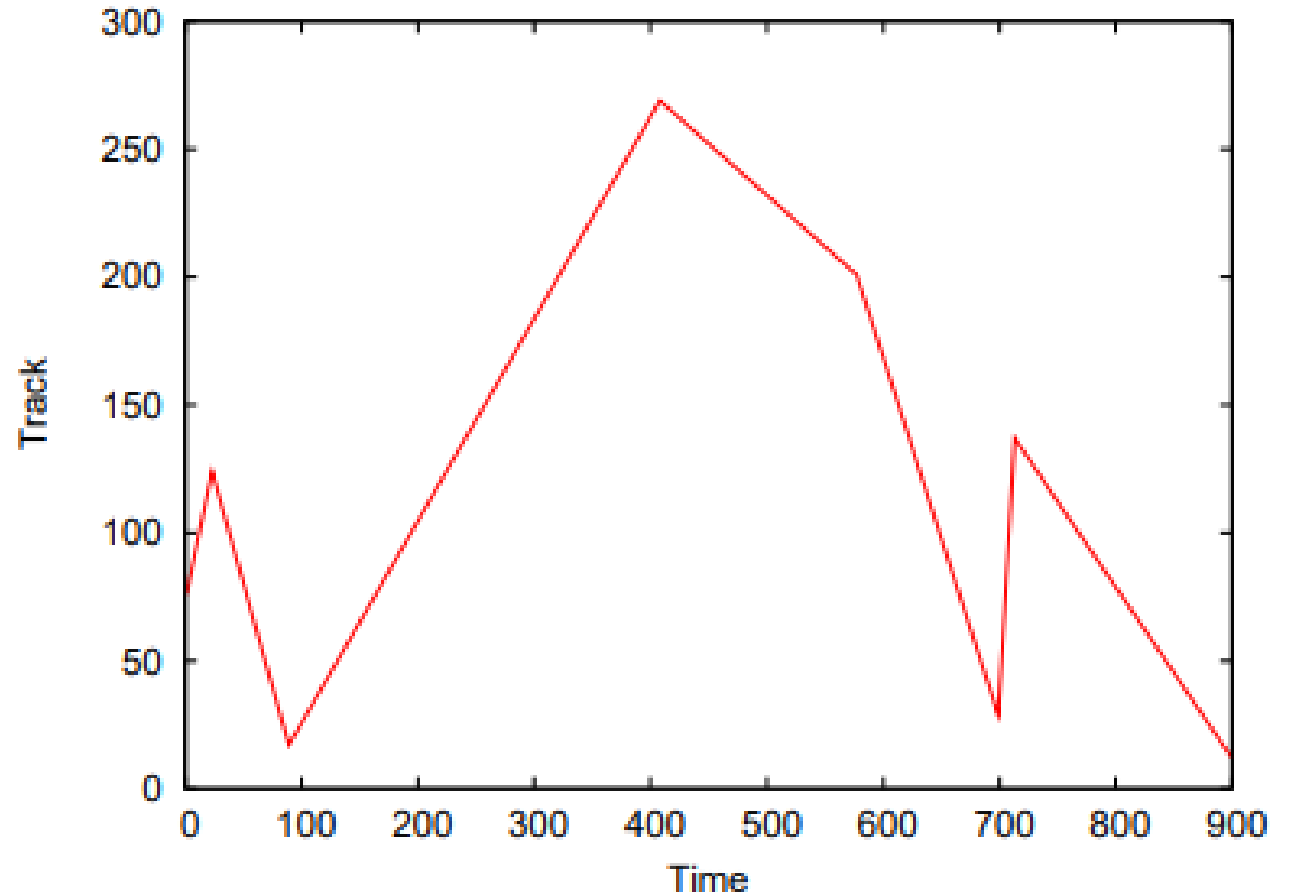


- La durée pour accéder à un secteur :
 - Attendre que le disque tourne jusqu'au bon angle
 - A 7200 rpm : 1 tour = 0.14 ms
 - Attendre que le bras se déplace jusqu'à la bonne piste :
beaucoup plus long : 5-20 ms !
 - C'est difficile d'aller très vite parce que le bras ne sait jamais où il devra aller ensuite

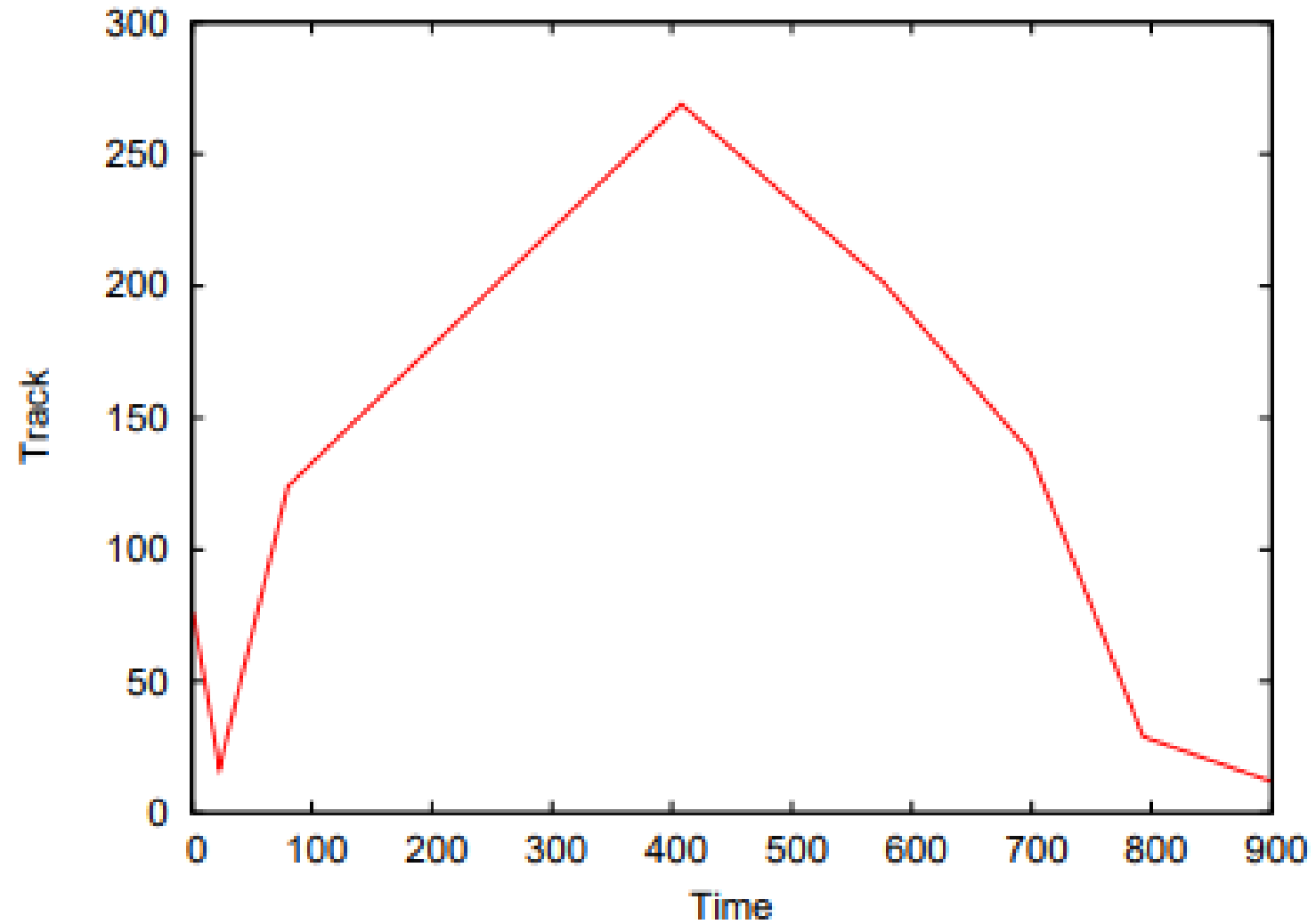
- Comment réduire les délais :
 1. Faire tourner le disque plus vite (pas beaucoup de gains à en attendre)
 2. Placer les données côte à côte (défragmentation)
 3. Ré-ordonnancer les accès quand on reçoit plusieurs requêtes pour des pistes (par ex : plusieurs utilisateurs accèdent à des fichiers différents en même temps)

FIRST-COME-FIRST-SERVED (FCFS)

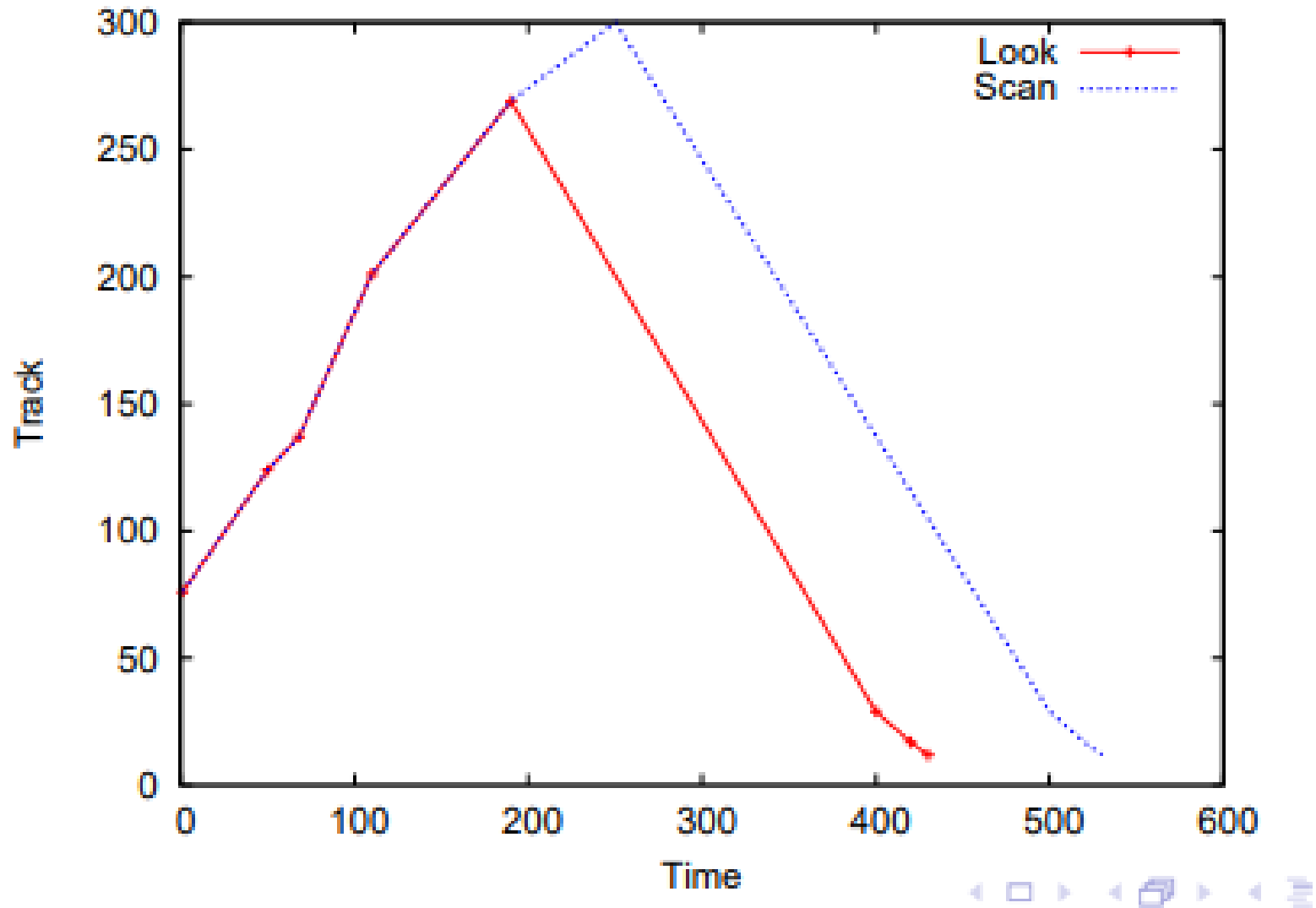
- Idée : traiter les requêtes dans l'ordre où elles arrivent
 - Simple
 - Beaucoup de déplacement du bras
- ! Mauvaise performance



- Idée : traiter la requête en attente la plus proche de la position actuelle
 - Moins de déplacements du bras → plus rapide
 - Sous forte charge : certaines requêtes peuvent ne jamais être traitée

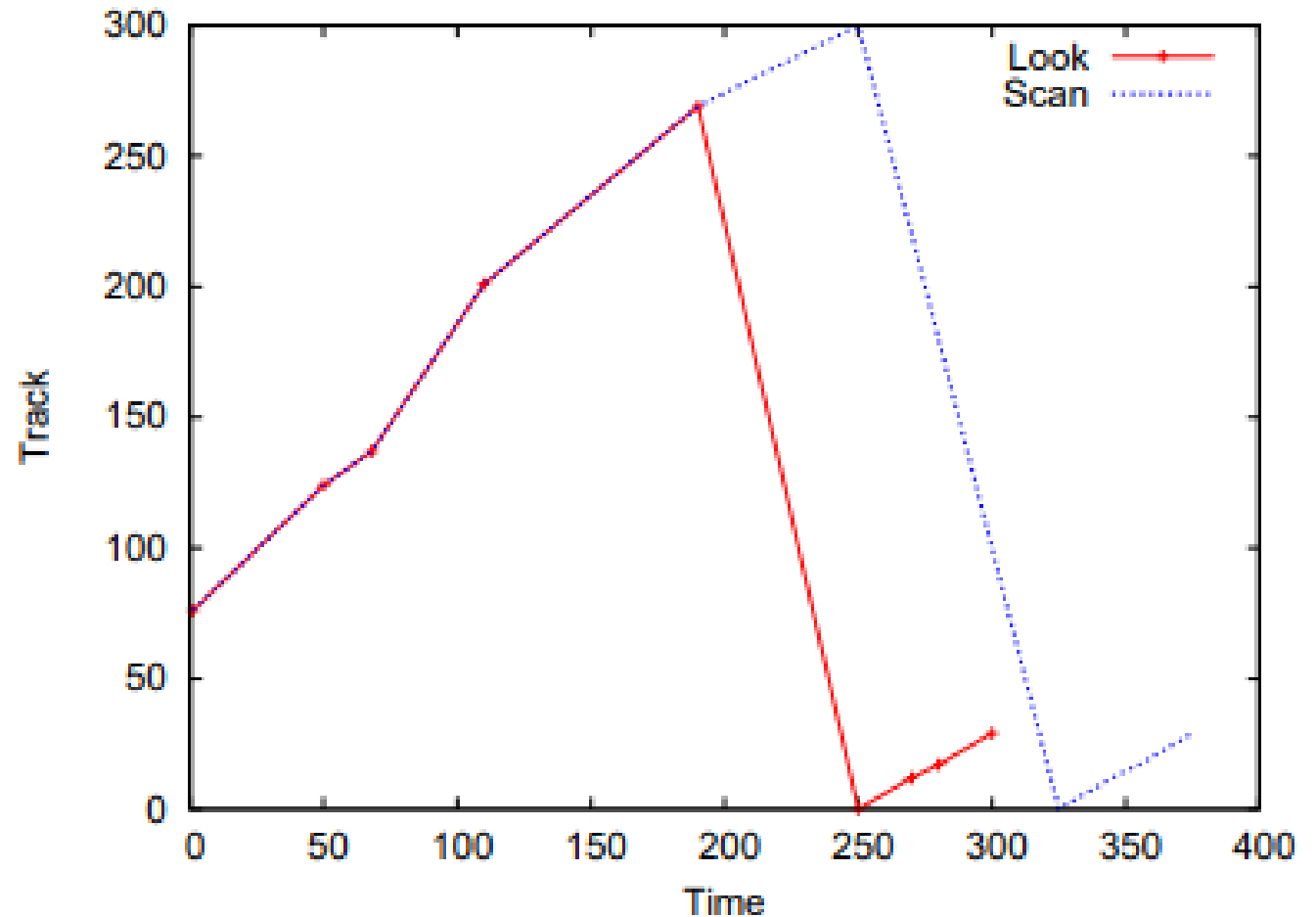


- Scan : déplacer le bras le long de toutes les pistes, traiter les requêtes pour les pistes devant lesquelles on passe
 - Toutes les requêtes sont traitées
 - Bonne performance sous forte charge
 - Temps d'accès parfois important !
- Look : changer de direction s'il n'y a plus de requête à traiter dans la direction actuelle



SCAN ET LOOK CIRCULAIRES

- Certains disques ont une fonction très efficace « ramener la tête à la piste 0 »
- Idée : **scan/look dans une seule direction**
 - Plus équitable entre les différentes requêtes



PLAN

INTRODUCTION

I. LES PÉRIPHÉRIQUES D'ENTRÉES/SORTIES

II. ORDONNANCEMENT DES ENTRÉES/SORTIES

III. LES DEADLOCKS

CONCLUSION

Un deadlock (ou impasse en français) est une situation qui se produit dans un système d'exploitation lorsqu'un ensemble de processus se bloque mutuellement, les empêchant de continuer leur exécution normale. Dans le contexte d'un cours de système d'exploitation, les deadlocks sont un sujet important car ils peuvent entraîner un dysfonctionnement grave du système.

Les caractéristiques clés d'un deadlock :

- **Attente circulaire** : Pour qu'un deadlock se produise, chaque processus impliqué doit attendre qu'un certain événement se produise, mais cet événement est retardé par un autre processus dans l'attente. Il se forme ainsi une boucle ou une attente circulaire, où chaque processus attend un autre processus de l'ensemble.

- **Possession de ressources** : Chaque processus dans un deadlock détient au moins une ressource (comme un fichier, une mémoire, un périphérique matériel, etc.) et en attend une autre. Cela signifie que les processus sont en compétition pour un ensemble limité de ressources.

- **Incapacité de libérer des ressources** : Les processus en deadlock sont incapables de libérer les ressources qu'ils détiennent, soit parce qu'ils ne peuvent pas terminer leur exécution, soit parce qu'ils refusent de libérer les ressources tant qu'ils n'ont pas obtenu celles qu'ils attendent.

- **Immobilisation mutuelle** : Les processus impliqués dans un deadlock restent bloqués indéfiniment, à moins que le système d'exploitation intervienne pour les libérer.

Les deadlocks sont dangereux car ils peuvent entraîner une paralysie complète du système, ce qui signifie que aucun des processus impliqués ne peut progresser.

Pour gérer les deadlocks, les systèmes d'exploitation utilisent des algorithmes et des techniques spécifiques, notamment :

- **Détection de deadlock** : Le système d'exploitation peut surveiller l'état du système pour détecter la présence d'un deadlock. Une fois détecté, il peut prendre des mesures pour le résoudre.

- **Prévention de deadlock** : Il s'agit de concevoir le système de telle manière qu'un deadlock ne puisse pas se produire. Cela peut impliquer la gestion des ressources de manière à éviter les conditions nécessaires à la formation d'un deadlock.

- **Évitement de deadlock** : Cette approche consiste à planifier l'allocation des ressources de manière à éviter les conditions qui pourraient conduire à un deadlock. L'algorithme du banquier est un exemple d'approche d'évitement de deadlock.

- **Libération forcée** : Dans certaines situations, le système d'exploitation peut être amené à libérer de force les ressources détenues par un processus bloqué afin de résoudre le deadlock. Cela peut entraîner la terminaison du processus en question.

La gestion des deadlocks est un aspect critique de la conception et de la gestion des systèmes d'exploitation, et elle est généralement couverte en détail dans les cours de système d'exploitation pour comprendre comment éviter, détecter et résoudre ces situations complexes.

- Certains périphériques peuvent réaliser **plusieurs actions à la fois**
 - Par ex: un disque dur gère des accès simultanés
- Mais beaucoup de ressources ne peuvent réaliser qu'une seule **opération à la fois!**
 - Imprimante
 - Clavier
 - Certaines données dans les tables du noyau
 - etc.

- Le système permet de **verrouiller** l'accès à une ressource, mais cela peut créer des **deadlocks**
- Exemple : deux programmes veulent scanner une image et l'écrire sur un CD-ROM

Programme 1	Programme 2
Verrouiller le scanner —	— Verrouiller le CD-ROM
Impossible de verrouiller le CD-ROM!	Impossible de verrouiller le scanner !

Pour avoir un deadlock, il faut **quatre conditions** :

1. **Exclusion mutuelle**

- Une ressource ne peut être verrouillée que par un seul processus à la fois

2. **Hold and wait**

- Un processus qui a verrouillé une ressource peut demander à verrouiller d'autres ressources

3. Pas de préemption

- Une fois qu'un processus a verrouillé une ressource, le système ne peut pas la reprendre de force

4. Attentes circulaires

- Il faut un cycle de processus qui s'attendent les uns les autres

- Solution #1 : Ignorer le problème complètement
- Ce n'est pas forcément une mauvaise solution!
 - Est-ce que les deadlocks arrivent souvent ?
 - Est-ce qu'ils sont très gênants ?

- Solution #2 : Détecter et corriger
- Explorer périodiquement le graphe des processus en attente de ressources
 - Si on détecte un cycle : choisir l'un des processus du cycle, le tuer
- Encore plus simple (et plus brutal) : si un processus est bloqué depuis 1 heure, alors on le tue

Pas très élégant. . .

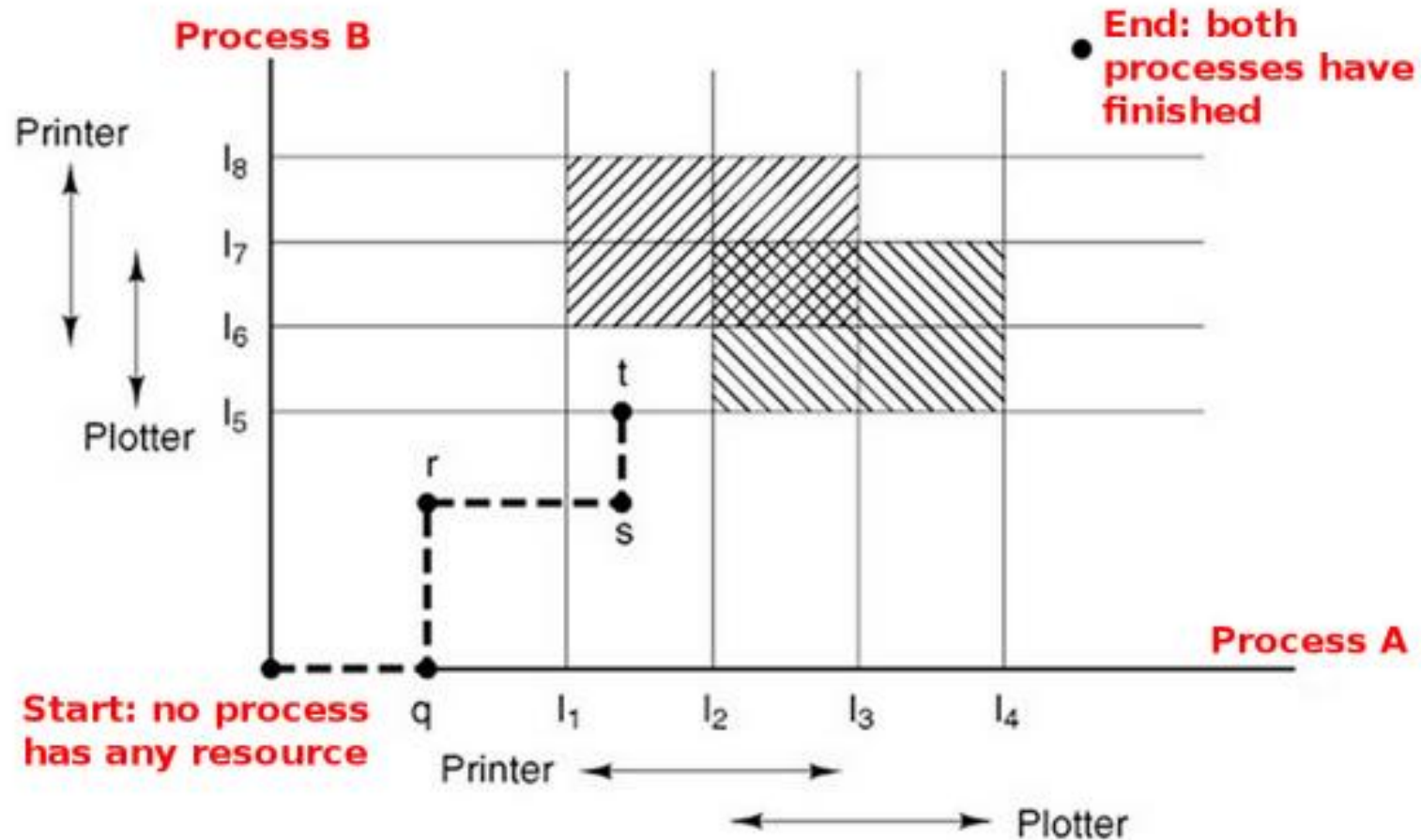
- Solution #3 : Empêcher les deadlocks de se produire

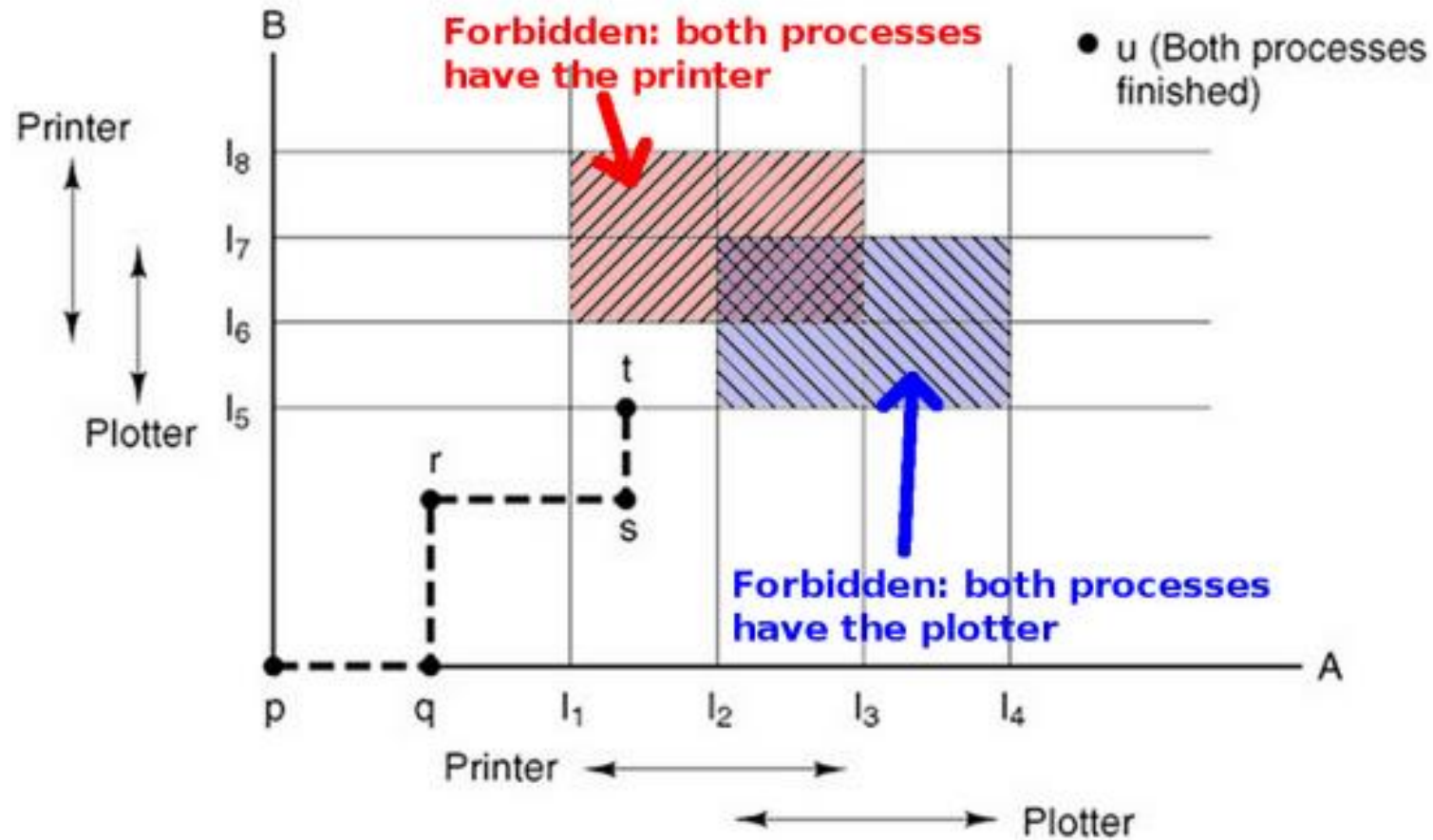
1. **Spooling** : faire en sorte que les processus n'aient pas besoin de verrouiller les ressources

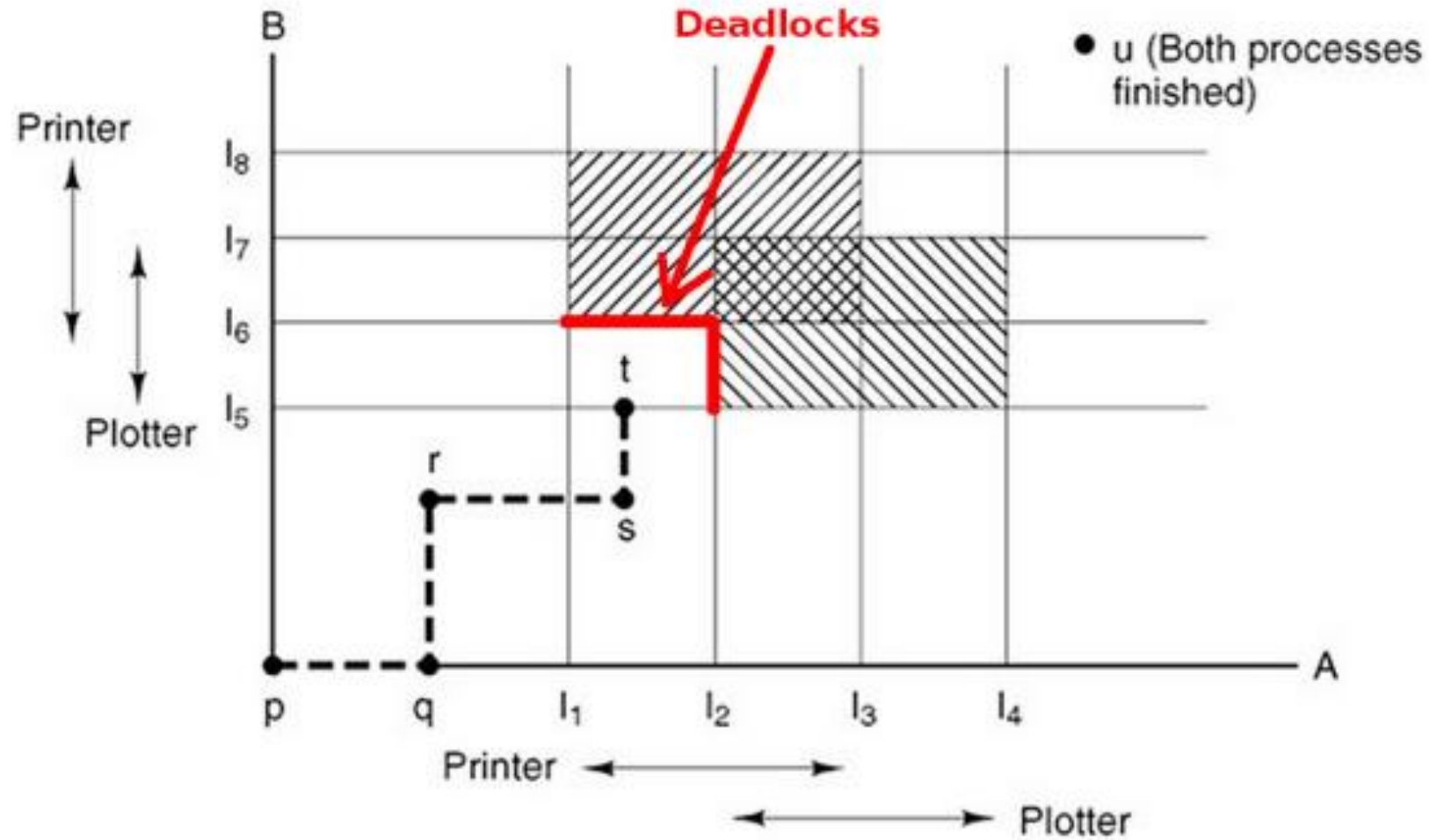
- Au lieu de laisser les processus verrouiller l'imprimante, on leur demande d'inscrire leurs fichiers à imprimer dans un « spool »
- Un seul processus peut verrouiller l'imprimante : le serveur d'impression
- Mais cela ne s'applique pas à toutes les ressources. Et que faire si le spool est plein ?

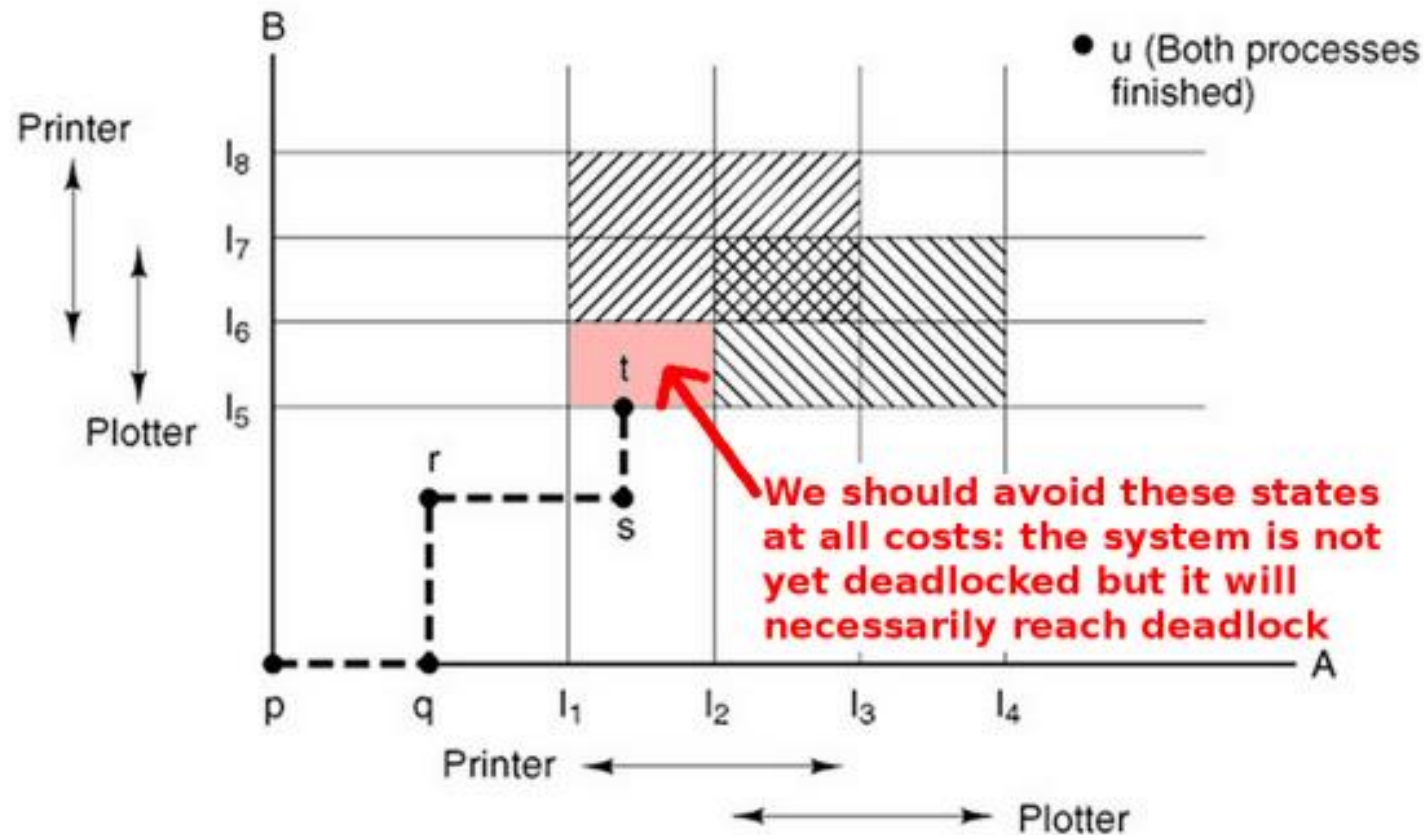
2. Verrouiller toutes les ressources nécessaires atomiquement
 - Pas facile à réaliser
3. Ordonnancer les ressources (p.ex en ordre alphabétique) et imposer que les processus ne verrouillent les ressources qu'en ordre croissant

- Solution #4 : **Eviter d'entrer en deadlock**: l'algorithme du banquier
 - Prévoir les **futures requêtes que d'autres processus pourraient faire**, et éviter d'entrer dans des états qui peuvent mener à des deadlocks









Cette technique suppose qu'on sait à l'avance quelles ressources peuvent être potentiellement verrouillées par quels processus

PLAN

INTRODUCTION

I. LES PÉRIPHÉRIQUES D'ENTRÉES/SORTIES

II. ORDONNANCEMENT DES ENTRÉES/SORTIES

III. LES DEADLOCKS

CONCLUSION

En conclusion, ce chapitre a abordé des aspects fondamentaux et essentiels de la gestion des **entrées/sorties** dans les systèmes informatiques. Nous avons exploré en profondeur la manière dont les périphériques d'entrées/sorties interagissent avec les systèmes d'exploitation, ainsi que les stratégies d'ordonnancement pour optimiser leur utilisation. De plus, nous avons étudié les deadlocks, des situations complexes qui peuvent paralyser un système, et nous avons examiné les méthodes de détection et de prévention de ces impasses.

Il est crucial de comprendre ces concepts pour concevoir, mettre en œuvre et maintenir des systèmes informatiques fiables et performants. La gestion efficace des périphériques d'entrées/sorties et la prévention des deadlocks sont des compétences essentielles pour les administrateurs système, les développeurs de logiciels et tous ceux qui travaillent dans le domaine de l'informatique.